

An Overview of Your Project Workspace

For a visual demonstration of the material in this document, please see <https://youtu.be/GsjvA9CX33Q>

In this demo, you will be working with different EDA tools that get design and technology information as inputs and export various files and reports after performing their tasks. Many of these exported files are imported as inputs to other tools and yet other files and design parameters are common to several tools.

Therefore, following many years of experience (and playing around with this constantly), I am providing you with a methodology for managing your files and workspaces for running digital implementation flows.

1 Your Project Directory Structure

Each project will reside inside a separate directory. I will refer to this as your **Project Root**.

Under your Project Root, my suggestion is to create several subdirectories, as illustrated in *Figure 1*, and described hereafter:

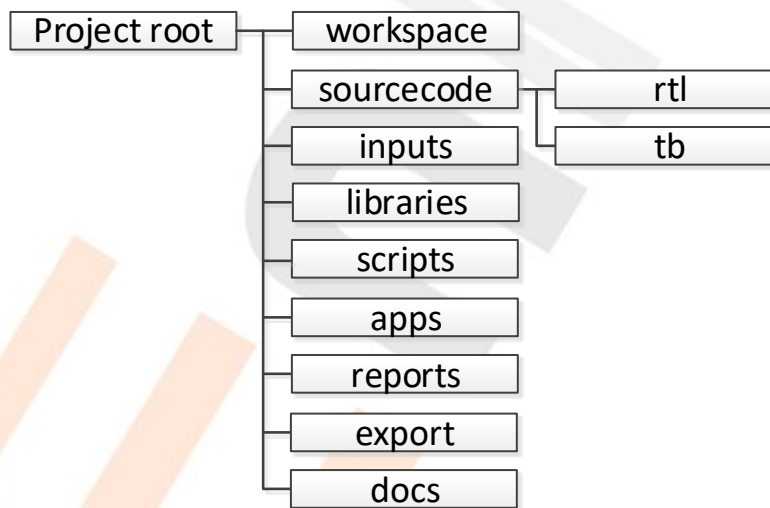


Figure 1: Suggested Workspace Setup

- **workspace**: The workspace directory is where you run the tools from. The concept is that all tools make a lot of clutter (log files, temporary files, etc.) and we want to keep this clutter separate from our “golden” files. Therefore, we make a parallel level directory and reference all our scripts, inputs, and output files by going back one level (`../`).

Creating the workspace in this way has several advantages:

- You can simply delete all the files in the workspace (`rm -rf workspace/*`) to clean up clutter without accidentally erasing important files.

- You can save all the logfiles and clutter of a certain run and start an additional run without mixing up the two by just creating another parallel directory (e.g., [workspace2/](#)). Since all of your other files are one directory up, there is no need to change anything in the scripts.
- You really don't need a separate directory for running each tool (i.e., simulation, synthesis, backend, etc.) and then you will be "running around" between directories a lot less.
- [sourcecode](#): This is where you store all your RTL files. One subdirectory ([rtl](#)) will store all your RTL files (or links to them). Another subdirectory ([tb](#)) will store all your Testbench files. You can add other subdirectories (such as included files) if you want.
- [inputs](#): This is where you will store or link to the inputs to your tools, such as design definitions, SDC files, etc. Many of these are shared between tools, so it is very convenient to have them all in the same subdirectory.
- [libraries](#): This is where the library definition scripts should be placed
- [scripts](#): This is where you will store your scripts for running your tools. You won't have *that many* separate script files, so you can conveniently put scripts for all tools in the same directory.
- [apps](#): This is where application files should be stored.
- [reports](#): This is where you will dump your reports for different stages of the flow.
- [export](#): This is where you will dump your exports, such as netlists, saved states, etc.
- [docs](#): This directory has documentation about the project and process.

2 Design Specific Input Files

The design specific input files are all named `<TOPLEVEL>.<extension>`. The reasoning behind this is to make sure you actually edit each of these files for a new design, rather than just copying it as is from a previous project. The specific files are:

- `<TOPLEVEL>.defines` : This file has all the basic design definitions for this project. The vast majority of them are stored in the `$design()` array, as described in Section 4. Since many of the definitions inside this file rely on the TOPLEVEL name of the design, the `$design(TOPLEVEL)` variable has to be set before reading this file.
- `<TOPLEVEL>.sdc` : This is the SDC file to be used with this project. It should be common to all stages of the project, such that if stage specific settings are required, they are applied with "if" statements. This will eliminate inconsistencies when updating the SDC in one tool and forgetting to propagate it to the other tools.
- `<TOPLEVEL>.mmmc`: This file has all the MMMC definitions used by Cadence tools (Genus, Innovus, etc.) for this project.
- `<TOPLEVEL>.cpf`: The Common Power Format (CPF) file that is shared by all tools (simulation, synthesis, P&R, LEC).
- `TOPLEVEL.io`: The I/O definition file to provide a location for the toplevel I/Os.
- `<TOPLEVEL>.floorplan.defines` : This file has floorplan specific design definitions for this project.
- `<TOPLEVEL>.ccopt` : This file has all clock tree synthesis definitions for the CCOpt tool.

Note that in the framework of this introductory course, we will not use a CPF file and we will not use an MMMC file during synthesis.

3 Library Specific Definition Files

The library specific input files are named `libraries.<library name>.tcl` and they are common to all projects that use this technology or library. Each technology (PDK) should have a base file and each library (e.g., standard cells, SRAMs, IOs, etc.) should have a separate library.

For example, the projects in this course we use the following library specific input files:

- `libraries.my-technology.tcl` : The library definitions for the PDK (technology only, not including standard cells, IOs, etc.).
- `libraries.my-standard-cells.tcl` : The library definitions for the Standard Cell libraries.
- `libraries.my-srams.tcl` : The library definitions for the SRAMs.
- `libraries.my-io-library.tcl` : The library definitions for the IO libraries.

4 TCL Arrays

Virtually all EDA tools use TCL as their shell language. I choose to use a number of basic TCL Arrays to store the primary information about my design, the implementation technology, the paths to the files, etc. This results in quite a bit more typing, when you're writing a script, but makes a the large number of variables a bit more manageable. It also has the following benefits:

- You minimize the chance that you will use the exact same name for two variables, which results in really nasty bugs.
- You provide some context for your variables, which helps reading and understanding code.
- You can easily pass variables to procedures by just passing the array variable as a global.

Note that I could have used namespaces, which are a really nice feature of TCL, but I find them hard to read and quite ugly, so I went for arrays instead.

The primary TCL Arrays that I use in the scripts that are provided to you in the course are:

- `$design()` : This array has all the data about this specific design. Most of the `$design()` members are defined in the `<TOPLEVEL>.defines` file. This includes the paths to files that are specific to the design.
- `$paths()` : This array has important paths in the Linux directory structure that help point to important files. Usually, updating the `$paths()` variables is enough when moving from one computing environment to another.
- `$tech_files()` : This array holds the specific files that are used for defining the technology, starting from the base technology (PDK) files, through library files, such as .lib and .lef files of standard cells, memories, etc.
- `$tech()` : This array holds values and parameters that are technology specific, such as the list of Clock Buffers or the minimum spacing between welltaps.

This document was provided to you courtesy of Prof. Adam Teman of the EnICS Labs Institute in the Alexander Kofkin Faculty of Engineering at Bar-Ilan University. The content was entirely written and compiled by Prof. Teman (without the help of AI tools), based on knowledge and non-proprietary data. You can find all of Prof. Teman's lectures and materials at the EnICS Labs website: <https://enicslabs.com/education/>

For any comments or enquiries, Prof. Teman can be reached at adam.teman@biu.ac.il